

从 Fork 到 PR

给别人的项目提代码，中间要跨越**三份仓库**和**两个 remote**。这张图把每一步在哪儿做、命令发给谁、冲突从哪冒出来，一次讲清楚。

先看全局：你手上其实有三份代码

大多数混乱都源于没分清这三份。命令发给谁，取决于你要动哪一份。



蓝色箭头是项目流向你（Fork、clone、fetch），**橙色箭头**是你的工作流向项目（push、PR）。虚线那条是 fetch：它只读取 upstream 的最新状态，不经过你的 fork，也不会改动任何东西。注意 upstream 不是 clone 时自动有的，得手动 `git remote add`。

六步

前两步是准备，中间两步是你的实际工作，后两步是往返。

01

GitHub 网页

Fork：在别人仓库里复制一份到你自己名下

打开原项目页面，点右上角 **Fork** → **Create fork**。你的账号下会多出一个同名仓库，这就是 **origin**。你有它的写权限，原项目没有。

坑 Fork 是 GitHub 网页上的功能，不是 git 命令。而且 fork 出来的仓库**不会自动跟原项目同步**，原项目更新了你的 fork 也不会变——这就是第 6 步存在的原因。

02

本地终端

Clone: 把你的 fork 拉到本地, 再挂上原项目

```
# 克隆的是你的 fork, 不是原项目
git clone git@github.com:你的用户名/项目名.git
cd 项目名

# clone 之后只有 origin 一个 remote
git remote -v

# 手动加上原项目, 习惯叫 upstream
git remote add upstream git@github.com:原作者/项目名.git

# 现在应该看到两个
git remote -v
```

坑 clone 错仓库是新手最常见的返工点: 克隆原项目的话你没有推送权限, 后面 git push 会直接失败。

03

本地终端

建分支: 别在 main 上改

```
git switch -c fix/login-retry
```

一个分支只做一件事, 名字带上类型前缀: feat/ 新功能、fix/ 修 bug、docs/ 文档。老的 git 用 git checkout -b, 作用一样。

坑 你 fork 的 main 唯一的用途是跟 upstream 对齐。在 main 上提交, 之后同步会一直打架。

04

本地终端

提交: 攒成一个干净的提交

```
git add 改动的文件
git commit -m "fix: 登录失败时重试一次"

# 看看自己相对 main 多了什么
git log --oneline main..HEAD
```

提交信息写「改了什么」, 比如 fix: 修复超时未重试。多个小提交也可以, PR 里能合并查看。

坑 提交前先写好 `.gitignore`。密钥、token、数据集、node_modules 一旦提交进历史很难彻底清掉。万一推上去了，改文件再提交是没用的，历史里还在。

05

本地终端

GitHub 网页

Push 上去，然后开 PR

```
git push -u origin fix/login-retry
```

推完终端会打印一条 GitHub 链接，点开就是 Compare 页面。或者回仓库页面点 **Compare & pull request**。

```
base:    原作者/项目名  ← main      (合到哪里)
compare: 你的用户名/项目名 ← fix/login-retry (拿什么合)
```

标题写清楚做了什么，描述里说**为什么这么改**、怎么验证的。开完把 PR 链接记下来，后面每次 push 到同一个分支，PR 都会自动更新。

坑 base 别选成你自己 fork 的 main。选错了 PR 会指向你自己的仓库，等于自己给自己提，原项目维护者看不到。

同步：把 upstream 的新提交接过来

你开工的这几天，原项目可能已经前进了。PR 要在**最新的 main 之上**才能干净地合进去，所以先拉下来。

```
# 只下载，不改动你的工作区
git fetch upstream

# 看看别人比你多了哪些提交
git log --oneline HEAD..upstream/main

# 把你的提交搬到 upstream 顶端（二选一，见下）
git rebase upstream/main
# 或 git merge upstream/main

# rebase 改写了历史，必须用这个推
git push --force-with-lease
```

推上去之后 PR 自动刷新，冲突也在这时候暴露出来。

🔄 只要 PR 还没被合并，**upstream 每动一次，第 6 步就要重做一次**。这是循环，不是一次性动作。PR 被合掉之后，把 fork 的 main 也同步一下：`git switch main && git fetch upstream && git merge upstream/main && git push`。

冲突：从哪来，怎么解

冲突不是错误，是 git 遇到了它没法替你决定的情况。



两边从同一个提交分叉，各自都改了同一文件的同一行。git 能自动合并改的是不同位置的改动，碰到同一行就只能停下来问你。注意**冲突发生在你要把 feat 搬到 upstream 顶端的那一刻**——不是你写代码写错了。

git 停下来的时候，会往文件里塞这样的标记：

```
<<<<<< HEAD
    timeout = 30
=====
    timeout = 60
>>>>>> fix/login-retry
```

坑 这是全流程里最容易记反的地方。 两半分别是谁，取决于你用的命令：

- `git merge upstream/main` 时，上半边是你自己的分支，下半边是 `upstream`；
 - `git rebase upstream/main` 时正好反过来——`git` 先把你的分支挪到 `upstream` 顶端，再一个重放你的提交，所以上半边是 `upstream`，下半边才是你的。
- 别背「上面是我的」，**看标记后面的名字**，那才是唯一的依据。

- 1 **看哪些文件冲突：** `git status`，标着 `both modified` 的那些。
- 2 **打开文件找到标记。** VS Code 会直接高亮，并给你 **Accept Current / Incoming / Both** 三个按钮，比手改快很多。
- 3 **把整段改成你想要的最终结果，然后删掉** `<<<<<<`、`=====`、`>>>>>>` **这三行**。标记留在文件里就是语法错误。
- 4 **标记解决完的文件：** `git add 文件名`。全部 `git add` 完才会继续。
- 5 **接着走完：** `rebase` 用 `git rebase --continue`，`merge` 用 `git commit`。中途所有提交都要重放一遍，可能连续冲突好几次——重复第 2 到 5 步就行。
- 6 **想整个放弃：** `git rebase --abort` 或 `git merge --abort`，会回到动手之前的状态，可以重来。

`git rebase upstream/main`

给开源项目提 PR 用这个

- 把你的提交搬到 `upstream` 顶端，历史是一条直线
- PR 的 diff 干净，维护者好读
- 代价：commit 的哈希被改写，推送必须 `--force-with-lease`

`git merge upstream/main`

自己长期维护的分支适合

- 多出一个合并提交，真实分叉被保留
- 不需要 `force push`，更安全
- 代价：PR 里会多出 `merge commit`，历史变乱

坑 推送改写过的分支只能用 `--force-with-lease`，不要用 `--force`。前者会在远端有你不知道的新提交时拒绝推送，后者会直接覆盖掉——万一你换过电脑、或者有别人一起在你的分支上干活，`--force` 会无声地删掉他们的东西。

命令速查

按执行顺序排的，左边是你要干的事。

要做什么	命令
把 fork 拉到本地	<code>git clone git@github.com:你的用户名/项目名.git</code>
挂上原项目	<code>git remote add upstream git@github.com:原作者/项目名.git</code>
确认两个 remote 都在	<code>git remote -v</code>
开新分支	<code>git switch -c fix/xxx</code>
提交	<code>git add . && git commit -m "fix: ..."</code>
推分支并建立追踪	<code>git push -u origin fix/xxx</code>
拉取原项目最新状态	<code>git fetch upstream</code>
看别人多了什么	<code>git log --oneline HEAD..upstream/main</code>
把改动接到最新 main 上	<code>git rebase upstream/main</code>
冲突时看状态	<code>git status</code>

要做什么	命令
解决完继续	<code>git rebase --continue</code>
放弃这次同步	<code>git rebase --abort</code>
推送改写后的分支	<code>git push --force-with-lease</code>
PR 合并后同步自己的	<code>git switch main && git fetch upstream && git merge upstream/main && git push</code>